## PORTAFOLIO TÉCNICO FINAL DE UNIDAD 1

# PROGRAMACIÓN ORIENTADA A OBJETOS (CDS-0201).

- NAYELI JAMILETH CÓRDOVA PIONCE

ENLACE: HTTPS://GITHUB.COM/NAYECORDOVA/P00-2N-ESPAM-NC.GIT

1 DE NOVIEMBRE DEL 2026

# ANÁLISIS DEL PROBLEMA:

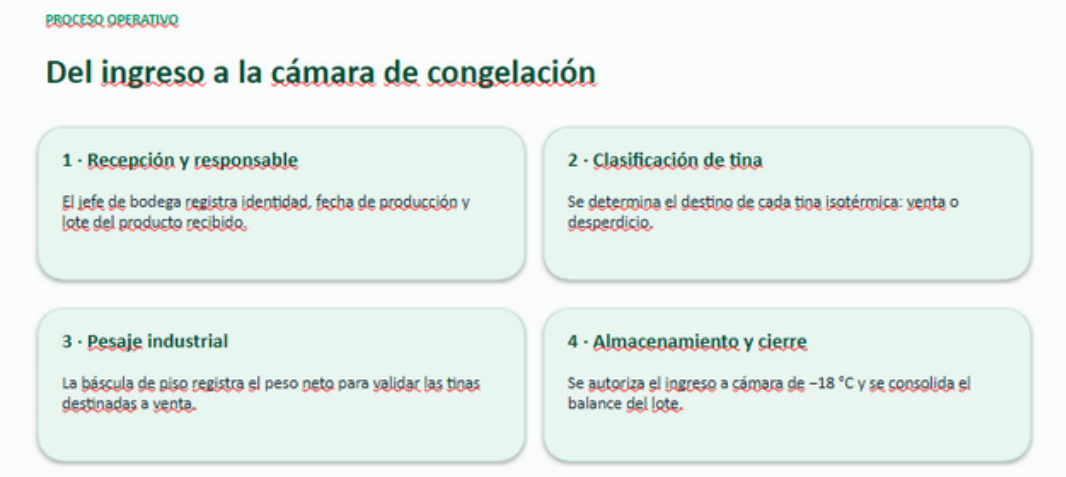



La abstracción del sistema se fundamenta en modelar el proceso físico real de recepción e ingreso de tina atunera en la planta de procesamiento como entidades de software dentro del paradigma orientado a objetos.

**Las entidades físicas clave del proceso real identificadas en la planta son:**

- **Tina:** Representa el contenedor o carga física de atún procesado. Se modela mediante sus atributos indispensables: código identificador , peso registrado en báscula (kg), estado de clasificación (Tina Completa: 600-605[ kg], Incompleta: <600\kg], Excedente: >605\ kg]) y temperatura de almacenamiento en cámara congeladora (-18^)
- **Ingreso a Bodega:** Modela el flujo lógico desde la descarga, verificación por el Jefe de Bodega, pesado digital en báscula industrial y clasificación, hasta la validación de reglas de negocio para el cierre de ingreso en el sistema.

El modelo encapsula el estado interno del lote y sus validaciones (invariantes como límites de peso e identificadores válidos) para evitar inconsistencias operativas antes del ingreso final a la cámara de congelación.



**Redacción según la técnica de presentación SCQA (Situación, Complicación, Pregunta, Respuesta):**

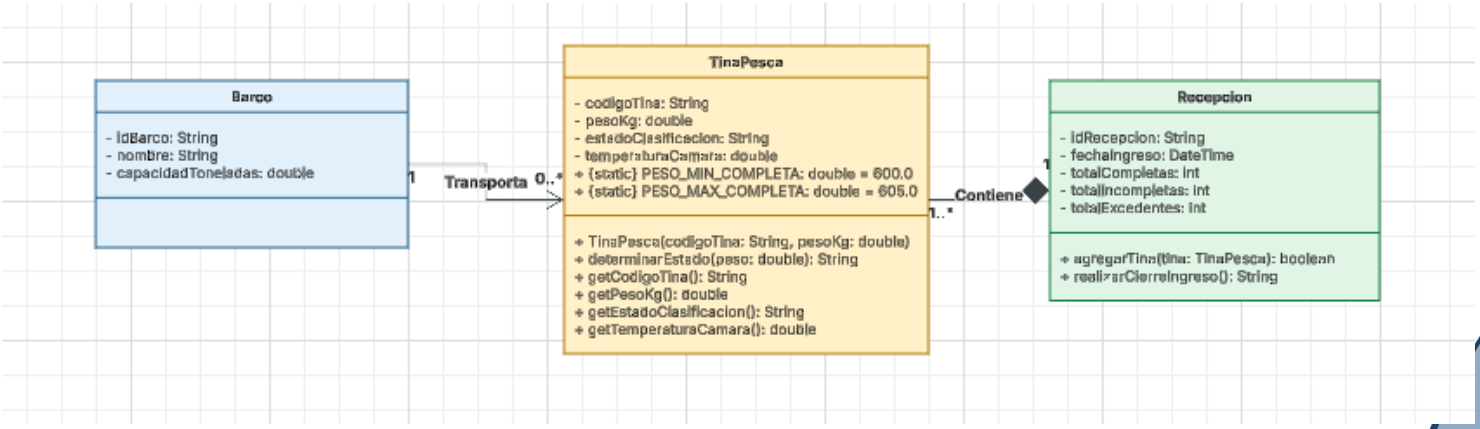
**Situación:** La planta atunera recibe lotes y tinas de pesca en bodega para su clasificación, pesado en báscula industrial de piso y posterior congelamiento a -18^

- **Complicación:** Durante la recepción física, existe el riesgo de registrar tinas con pesos fuera de rango (incompletas o con excedente) o códigos de identificación no válidos, lo que puede causar fallas en la trazabilidad de la mercancía y registros erróneos en el sistema informático.

**Pregunta:** ¿Cómo estructurar un modelo Orientado a Objetos en Java/Dart que abstraiga correctamente el proceso físico de recepción, garantice la integridad de los datos mediante validaciones de invariantes y permita la migración portable a otros lenguajes manteniéndose sólido y extensible?

- **Respuesta:** Se diseña un modelo de dominio clases (Recepcion, Tina) que encapsula la lógica de negocio (reglas de peso, control de estados de la tina), manejando excepciones controladas ante datos inválidos y separando la capa de dominio de la consola de ejecución en Java y Dart.

# DIAGRAMA DE CLASES




**Barco 1 ------- 0.. TinaPesca (Asociación / Agregación):**

- **Multiplicidad:** Un Barco puede transportar y desembarcar de 0 a muchas (0..) TinaPesca. Una TinaPesca proviene de 1 Barco específico.
- **Visibilidad:** El acceso a los datos de la tina se restringe mediante encapsulamiento privado (-), exponiendo solo getters/setters públicos (+).

**Recepcion 1 <>------- 1.. TinaPesca (Composición):**

- **Multiplicidad:** Una Recepción administra de 1 a muchas (1..) TinaPesca. Cada TinaPesca pertenece a un proceso de ingreso/recepción en bodega.
- **Visibilidad y Contrato:** Recepcion mantiene sus contadores internos (totalCompletas, etc.) en privado (-) y expone métodos públicos (+) para registrar tinas y emitir el resumen de cierre.



# CÓDIGO BODEGA

Main.java src M

IngresoBodega.java > IngresoBodega

```java
public class IngresoBodega {

    public void registrarTina(Tina tina) {
        loteTinas.add(tina);
    }

    public void generarInforme() {
        int completas = 0;
        int incompletas = 0;
        int excedentes = 0;
        int desperdicios = 0;

        for (Tina t : loteTinas) {
            String estado = t.evaluarEstado();
            if (estado.equals("Tina Completa")) {
                completas++;
            } else if (estado.equals("Tina Incompleta")) {
                incompletas++;
            } else if (estado.equals("Tina con Excedente")) {
                excedentes++;
            } else if (estado.equals("Desperdicio")) {
                desperdicios++;
            }
        }

        System.out.println("\n--- INFORME DE CIERRE DE BODEGA ---");
        System.out.println("Tinas Completas: " + completas);
        System.out.println("Tinas Incompletas: " + incompletas);
        System.out.println("Tinas con Excedente: " + excedentes);
        System.out.println("Desperdicios: " + desperdicios);
    }
```

# CÓDIGO TINA

```java
    public Tina() {
        this.tipoDestino = "Venta";
    }

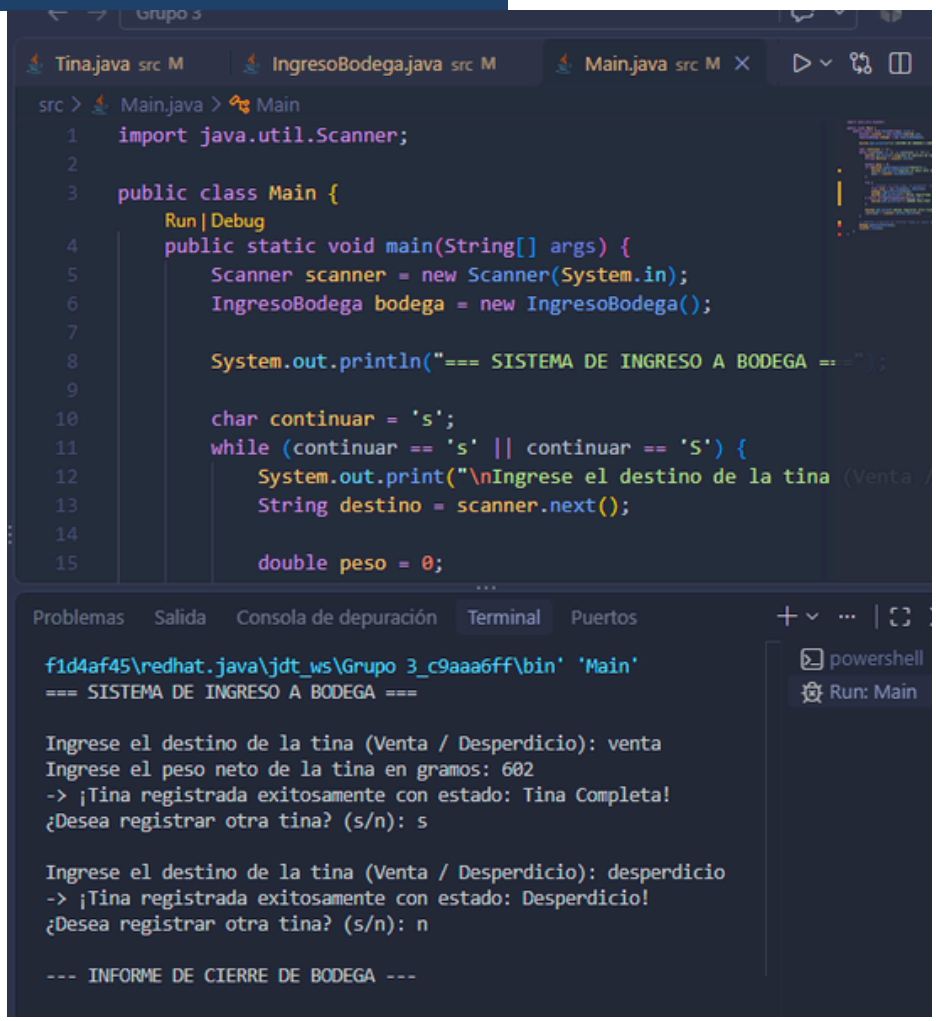
    // 2. Constructor principal con validación Fail-Fast y des
    public Tina(double pesoNeto, String tipoDestino) {
        // Validación Fail-Fast: Lanza excepción si los datos
        if (tipoDestino == null || tipoDestino.trim().isEmpty()) {
            throw new IllegalArgumentException("El tipo de destino no
        }
        if (pesoNeto < 0) {
            throw new IllegalArgumentException("El peso neto no puede
        }

        this.pesoNeto = pesoNeto;
        this.tipoDestino = tipoDestino;
    }

    // 2. Constructor sobrecargado usando encadenamiento con this
    public Tina(String tipoDestino) {
        this(pesoNeto: 600.0, tipoDestino); // Llama al constructor de
    }

    // Método para evaluar el estado de la tina
    public String evaluarEstado() {
        if (tipoDestino.equalsIgnoreCase("Desperdicio")) {
            return "Desperdicio";
        }
        if (pesoNeto >= 600 && pesoNeto <= 605) {
            return "Tina Completa";
        } else if (pesoNeto < 600) {
```

# CÓDIGO EJECUTADO




**Elección de Dart:** Se selecciona Dart dada su fuerte orientación a objetos, tipado estático opcional y soporte nativo para constructores nombrados/redireccionados.

**Equivalencias de sintaxis con respecto a Java:**

- **Encapsulamiento:** En Java se usan modificadores private; en Dart se usa el prefijo de guion bajo _ (ej. _codigoTina).
- **Constructores sobrecargados:** Java sobrecarga métodos por firma de argumentos; Dart utiliza constructores nombrados (ej. TinaPesca.conTemperatura()).
- **Invariantes y Excepciones:** Ambas sintaxis lanzan excepciones de argumento (ArgumentError en Dart equivale a IllegalArgumentException en Java).



```dart
void main() {
  IngresoBodega bodega = IngresoBodega();

  print('=== SISTEMA DE INGRESO A BODEGA (DART) ===');

  String? continuar = 's';
  while (continuar != null && continuar.toLowerCase() == 's') {
    stdout.write('\nIngrese el destino de la tina (Venta / Desperdicio): ');
    String destino = stdin.readLineSync() ?? 'Venta';

    double peso = 0;
    if (destino.toLowerCase() == 'venta') {
      stdout.write('Ingrese el peso neto de la tina en kg: ');
      peso = double.parse(stdin.readLineSync() ?? '0');
    }

    // Instanciación del objeto Tina y registro en bodega
    Tina tina = Tina(peso, destino);
    bodega.registrarTina(tina);

    stdout.write('¿Desea registrar otra tina? (s/n): ');
    continuar = stdin.readLineSync();
  }

  // Generación del reporte final
  bodega.generarInforme();
```

# CÓDIGO MAIN DART

# CÓDIGO TINA DART

Tina.dart

```dart
tina.Dart
import 'dart:io';

// 1. Clase Tina
class Tina {
  // Atributos privados (en Dart el prefijo '_' indica privac
  double _pesoNeto;
  String _tipoDestino; // "Venta" o "Desperdicio"

  // Constructor
  Tina(this._pesoNeto, this._tipoDestino);

  // Método que evalúa el estado de la tina
  String evaluarEstado() {
    if (_tipoDestino.toLowerCase() == 'desperdicio') {
      return 'Desperdicio';
    }

    // Reglas de la báscula de plataforma
    if (_pesoNeto >= 600 && _pesoNeto <= 605) {
      return 'Tina Completa';
    } else if (_pesoNeto < 600) {
      return 'Tina Incompleta';
    } else {
      return 'Tina con Excedente';
    }
  }
}
```

# CONCLUSIONES

La aplicación del paradigma orientado a objetos permitió abstraer fielmente el proceso físico de recepción en la planta atunera, protegiendo las reglas de negocio (pesos y temperaturas de congelación) mediante el control de invariantes de clase para asegurar la integridad de los datos en todo momento.
El diseño estructurado de las clases demostró una alta portabilidad al migrar la lógica de Java a Dart sin alterar la semántica ni el comportamiento del sistema, garantizando un manejo de excepciones consistente y extensible para entornos industriales.

- El paradigma Orientado a Objetos (POO) proporciona un marco conceptual sumamente sólido para la modelación de procesos industriales complejos, como el ingreso y clasificación de mercancía en una planta atunera.

- diseñar software industrial bajo los principios de POO y validación de invariantes no solo facilita el mantenimiento y la extensibilidad del sistema a futuro, sino que también asegura que los comportamientos esperados en la consolas de ejecución de distintos entornos tecnológicos produzcan resultados completamente consistentes y robustos.